Apache Airflow

Workflow orchestration at scale

Open-source platform to **author, schedule, and monitor** workflows programmatically in Python. Created at Airbnb in 2014 — now an Apache top-level project used everywhere from startups to banks. Core philosophy: **configuration as code** — pipelines are Python files, not drag-and-drop UIs.

Configuration as code | Python-native | Batch workflows | Extensible operators | Version-controllable

THE CORE CONCEPT — WHAT IS A DAG?

Tasks run in order. Task B cannot run before Task A finishes. No circular dependencies allowed.

CORE CONCEPTS

DAG

The full workflow definition — a Python file describing what tasks to run, in what order, and when.

Task

One unit of work inside a DAG. Each task is defined by an Operator and runs independently.

Operator

The worker template that defines what the task does — run Python, execute SQL, call an API, etc.

Scheduler

The brain — polls DAGs and decides which tasks are ready to run based on schedule and dependencies.

DAG Run

One execution instance of a DAG. Each run has a unique execution_date and its own task states.

XCom

Cross-communication — lets tasks pass small data to each other (file paths, IDs). Not for large datasets.

AIRFLOW ARCHITECTURE — 6 COMPONENTS

Web Server
Flask-based UI. View DAG status, trigger runs, inspect logs, manage connections. Reads from the Metadata DB.

Scheduler
Runs continuously. Parses DAG files, decides which task instances are ready, hands them to the Executor.

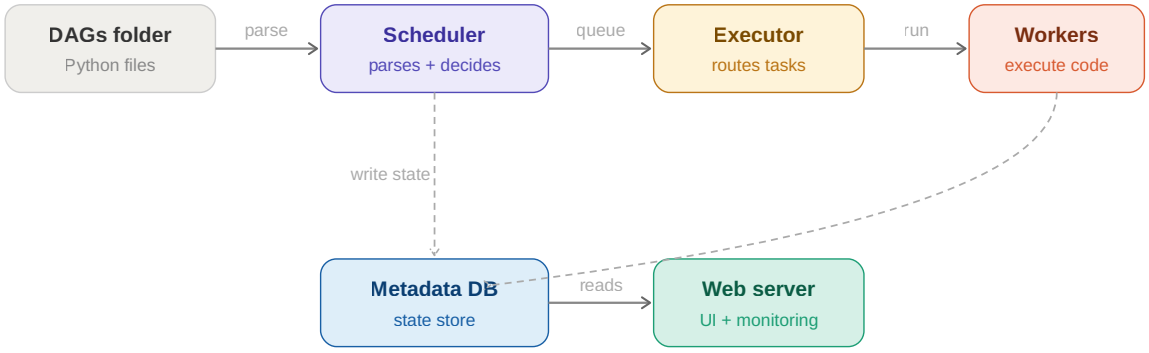
Executor
Part of the Scheduler (not a separate process). Routes tasks to workers. Local, Celery, or Kubernetes flavors.

Workers
Separate processes that actually execute the task code. Can scale horizontally based on workload.

Metadata DB
Postgres (recommended) or MySQL. Stores all DAG runs, task states, connections, variables, logs, and XComs.

DAGs Folder
Directory of Python files. Scheduler scans it periodically. Web server, scheduler, and workers all need access.

ARCHITECTURE DATA FLOW



Workers update task status → Metadata DB → Web server reads for UI display

OPERATOR TYPES

PythonOperator
Calls any Python callable. Most flexible — run ETL logic, ML scoring, anything Python can do.

BashOperator
Executes a shell command. Great for CLI scripts, shell ETL, or calling external tools.

PostgresOperator
Runs SQL against a Postgres connection. MySQL, Hive, BigQuery variants also available.

Sensors
Wait for a condition (file appears, API ready, time reached) before downstream tasks run.

DockerOperator
Spins up a Docker container to run a task in a fully isolated environment.

KubernetesPodOperator
Launches a pod on Kubernetes per task. The cloud-native production pattern at scale.

SparkSubmitOperator
Submits a Spark job. Core to modern lake-house data engineering pipelines.

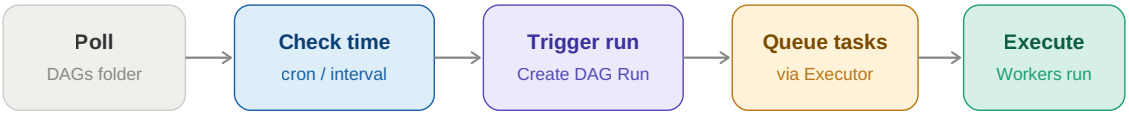
EmailOperator
Sends alert emails. Commonly used in on_failure_callback patterns for monitoring.

TASK INSTANCE LIFECYCLE — STATES

No status	Scheduled	Queued	Running
Success ✓	Failed ✗	Retrying	Skipped

> Scheduler decides the state. Worker does the work. The metadata DB remembers everything.

HOW SCHEDULING WORKS — STEP BY STEP



schedule_interval accepts cron strings ("0 6 * * *"), timedelta objects, or "@daily" / "@hourly" presets

WRITING A DAG — MINIMAL ETL PATTERN

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

def extract(): print("Pulling data...")
def transform(): print("Cleaning data...")
def load(): print("Writing to warehouse...")

with DAG("etl_pipeline",
     start_date=datetime(2024, 1, 1),
     schedule_interval="@daily",
     catchup=False) as dag:

  t1 = PythonOperator(task_id="extract", python_callable=extract)
  t2 = PythonOperator(task_id="transform", python_callable=transform)
  t3 = PythonOperator(task_id="load", python_callable=load)

  t1 >> t2 >> t3   # dependency chain: left runs before right
```

EXECUTOR COMPARISON

SequentialExecutor

One task at a time. Dev/testing only. Uses

LocalExecutor

Parallel tasks via subprocess on one machine. Good for

CeleryExecutor

Distributed workers via Redis / RabbitMQ. The

KubernetesExecutor

One pod per task. Fully isolated, auto-scales.

SQLite. Zero parallelism.	small production setups + Postgres.	go-to for multi-machine production clusters.	Preferred for cloud-native DE platforms.

COMMON USE CASES

📥 ETL pipelines

Extract → transform → load on a schedule. The core Airflow use case at every data org.

🧠 ML pipelines

Feature engineering → model training → evaluation → deployment → monitoring.

📊 Reporting

Nightly aggregations → dashboard refresh → email summaries to stakeholders.

☁️ Data lake loads

Ingest from APIs, S3, Kafka into lake-house tables like Delta or Iceberg.

🔁 Dependency chains

Job B starts only after Job A writes a success file — the sensor pattern.

🛡️ Data quality

Post-load validation checks, anomaly detection, SLA monitoring and alerting.

Best practice: Keep DAGs declarative — define the pipeline shape in the DAG file, call external scripts or operators for logic. Never import heavy dependencies at module level — it slows scheduler parsing.

Staff-level insight: The Scheduler is stateless by design — it re-parses DAGs from scratch on every scan. Fast parsing = healthy cluster. Slow DAG files are the #1 Airflow performance issue in production.